Digital design intern ADI

AHB lite protocol v1 master design

Made by:

Hagar Emadeldin Fathy Emara

Submitted to:

Eng/ Fatma Ali

AHB lite protocol master introduction:

AHB-Lite implements the features required for high-performance, high clock frequency systems. It is a bus interface that supports a single bus master and provides high-bandwidth operation.

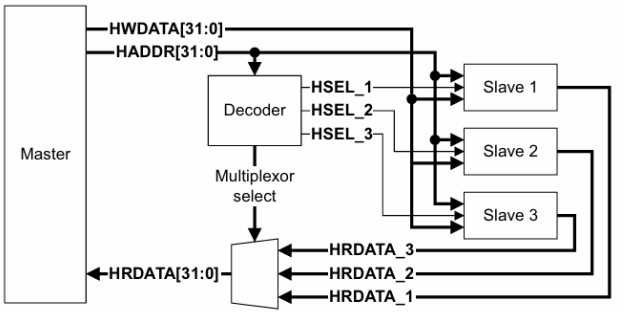
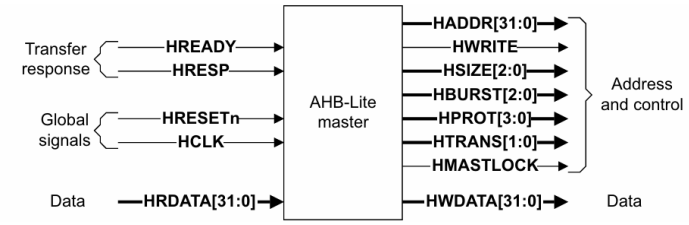


Figure 1-1 AHB-Lite block diagram

The most common AHB-Lite slaves are internal memory devices, external memory interfaces, and high bandwidth peripherals.

Figure 1-1 shows a single master AHB-Lite system design with one AHB-Lite master and three AHB-Lite slaves.

AHB master



The master starts a transfer by driving the address and control signals. These signals provide information about the address, direction, width of the transfer, and indicate if the transfer forms part of a burst. Transfers can be:

• single

• incrementing bursts that do not wrap at address boundaries

• wrapping bursts that wrap at particular address boundaries.

The write data bus moves data from the master to a slave, and the read data bus moves data from a slave to the master.

Every transfer consists of:

Address phase ---- one address and control cycle.

Data phase---- one or more cycles for the data.

A slave cannot request that the address phase is extended and therefore all slaves must be capable of sampling the address during this time. However, a slave can request that the master extends the

data phase by using HREADY. This signal, when LOW, causes wait states to be inserted into the transfer and enables the slave to have extra time to provide or sample data.

The slave uses HRESP to indicate the success or failure of a transfer.

Selected features and the unsupported ones:

Supported features in my design:

1. Single and INCR burst transfers.
2. The four transfer states IDLE, NONSEQ, SEQ, and BUSY.
3. Hsize of 8 byte, half word, and word.
4. Error response from slave (Hresp)
5. Waited transfers

Unsupported features:

1. Protection
2. Master lock
3. INCR4, INCR8, WRAP4…… and other burst operations
4. HSIZE above word.

The critical and useful info from the standard to make the design:

Basic transfers

An AHB-Lite transfer consists of two phases:

Address Lasts for a single **HCLK** cycle unless its extended by the previous bus transfer.

Data That might require several **HCLK** cycles. Use the **HREADY** signal to control the number of clock cycles required to complete the transfer.

HWRITE controls the direction of data transfer to or from the master. Therefore, when:

- **HWRITE** is HIGH, it indicates a write transfer and the master broadcasts data on the write data bus, **HWDATA[31:0]**
- **HWRITE** is LOW, a read transfer is performed and the slave must generate the data on the read data bus, **HRDATA[31:0]**.

HTRANS[1:0]	Type	Description
b00	IDLE	Indicates that no data transfer is required. A master uses an IDLE transfer when it does not want to perform a data transfer. It is recommended that the master terminates a locked transfer with an IDLE transfer. Slaves must always provide a zero wait state OKAY response to IDLE transfers and the transfer must be ignored by the slave.
b01	BUSY	The BUSY transfer type enables masters to insert idle cycles in the middle of a burst. This transfer type indicates that the master is continuing with a burst but the next transfer cannot take place immediately. When a master uses the BUSY transfer type the address and control signals must reflect the next transfer in the burst. Only undefined length bursts can have a BUSY transfer as the last cycle of a burst. See *Burst termination after a BUSY transfer* on page 3-10 for more information. Slaves must always provide a zero wait state OKAY response to BUSY transfers and the transfer must be ignored by the slave.
b10	NONSEQ	Indicates a single transfer or the first transfer of a burst. The address and control signals are unrelated to the previous transfer. Single transfers on the bus are treated as bursts of length one and therefore the transfer type is NONSEQUENTIAL.
b11	SEQ	The remaining transfers in a burst are SEQUENTIAL and the address is related to the previous transfer. The control information is identical to the previous transfer. The address is equal to the address of the previous transfer plus the transfer size, in bytes, with the transfer size being signaled by the **HSIZE[2:0]** signals. In the case of a wrapping burst the address of the transfer wraps at the address boundary.

HSIZE[2:0] indicates the size of a data transfer. Table 3-2 lists the possible transfer sizes.

Table 3-2 Transfer size encoding

HSIZE[2]	HSIZE[1]	HSIZE[0]	Size (bits)	Description
0	0	0	8	Byte
0	0	1	16	Halfword
0	1	0	32	Word
0	1	1	64	Doubleword
1	0	0	128	4-word line
1	0	1	256	8-word line
1	1	0	512	-
1	1	1	1024	-

Note

The transfer size set by **HSIZE** must be less than or equal to the width of the data bus. For example, with a 32-bit data bus, **HSIZE** must only use the values b000, b001, or b010.

Use **HSIZE** in conjunction with **HBURST**, to determine the address boundary for wrapping bursts.

The **HSIZE** signals have exactly the same timing as the address bus. However, they must remain constant throughout a burst transfer.

Burst termination after a BUSY transfer

After a burst has started, the master uses BUSY transfers if it requires more time before continuing with the next transfer in the burst.

During an undefined length burst, INCR, the master might insert BUSY transfers and then decide that no more data transfers are required. Under these circumstances, it is acceptable for the master to then perform a NONSEQ or IDLE transfer that then effectively terminates the undefined length burst.

The protocol does not permit a master to end a burst with a BUSY transfer for fixed length bursts of type:

- incrementing INCR4, INCR8, and INCR16
- or wrapping WRAP4, WRAP8, and WRAP16.

These fixed length burst types must terminate with a SEQ transfer.

The master is not permitted to perform a BUSY transfer immediately after a SINGLE burst. SINGLE bursts must be followed by an IDLE transfer or a NONSEQ transfer.

Early burst termination

Bursts can be terminated by either:

- *Slave error response*
- *Multi-layer interconnect termination* on page 3-11.

Slave error response

If a slave provides an ERROR response then the master can cancel the remaining transfers in the burst. However, this is not a strict requirement and it is also acceptable for the master to continue the remaining transfers in the burst.

If the master does not complete that burst then there is no requirement for it to rebuild the burst when it next accesses that slave. For example, if a master only completes three beats of an eight-beat burst then it does not have to complete the remaining five transfers when it next accesses that slave.

Transfer type changes during wait states

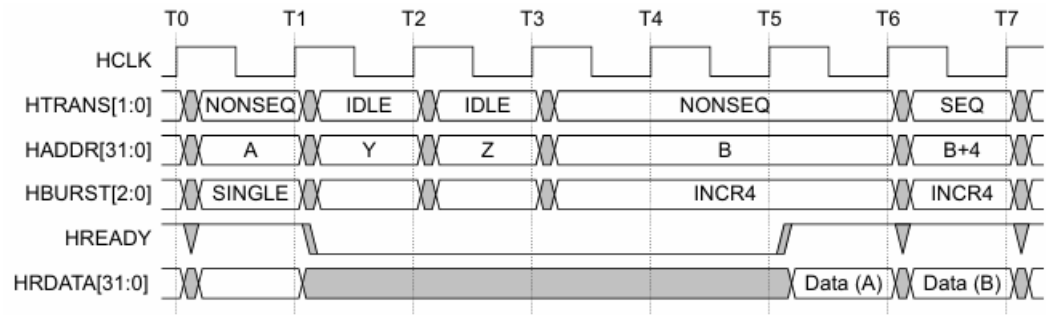
When the slave is requesting wait states, the master must not change the transfer type, except as described in:

- *IDLE transfer*
- *BUSY transfer, fixed length burst* on page 3-17
- *BUSY transfer, undefined length burst* on page 3-18.

IDLE transfer

During a waited transfer, the master is permitted to change the transfer type from IDLE to NONSEQ. When the **HTRANS** transfer type changes to NONSEQ the master must keep **HTRANS** constant, until **HREADY** is HIGH.

Figure 3-13 shows a waited transfer for a SINGLE burst, with a transfer type change from IDLE to NONSEQ.



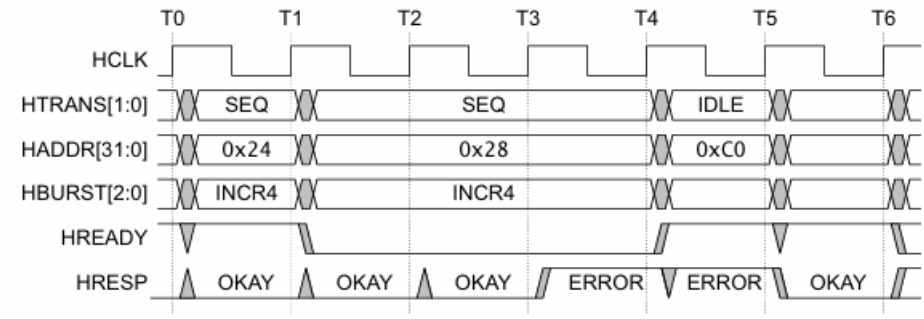
BUSY transfer, undefined length burst

During a waited transfer for an undefined length burst, INCR, the master is permitted to change from BUSY to any other transfer type, when **HREADY** is LOW. The burst continues if a SEQ transfer is performed but terminates if an IDLE or NONSEQ transfer is performed.

After an ERROR response

During a waited transfer, if the slave responds with an ERROR response then the master is permitted to change the address when **HREADY** is LOW. See *ERROR response* on page 5-3 for more information about the ERROR response.

Figure 3-17 shows a waited transfer, with the address changing following an ERROR response from the slave.



HRESP	Response	Description
0	OKAY	The transfer has either completed successfully or additional cycles are required for the slave to complete the request. The **HREADY** signal indicates whether the transfer is pending or complete.
1	ERROR	An error has occurred during the transfer. The error condition must be signaled to the master so that it is aware the transfer has been unsuccessful. A two-cycle response is required for an error condition with **HREADY** being asserted in the second cycle.

Table 5-1 shows that the complete transfer response is a combination of the **HRESP** and **HREADY** signals. Table 5-2 lists the complete transfer response based on the status of these two signals.

Table 5-2 Transfer response

	HREADY	
HRESP	**0**	**1**
0	Transfer pending	Successful transfer completed
1	ERROR response, first cycle	ERROR response, second cycle

Transfer done

A successful completed transfer is signaled when **HREADY** is HIGH and **HRESP** is OKAY.

Transfer pending

A typical slave uses **HREADY** to insert the appropriate number of wait states into the data phase of the transfer. The transfer then completes with **HREADY** HIGH and an OKAY response to indicate the successful completion of the transfer.

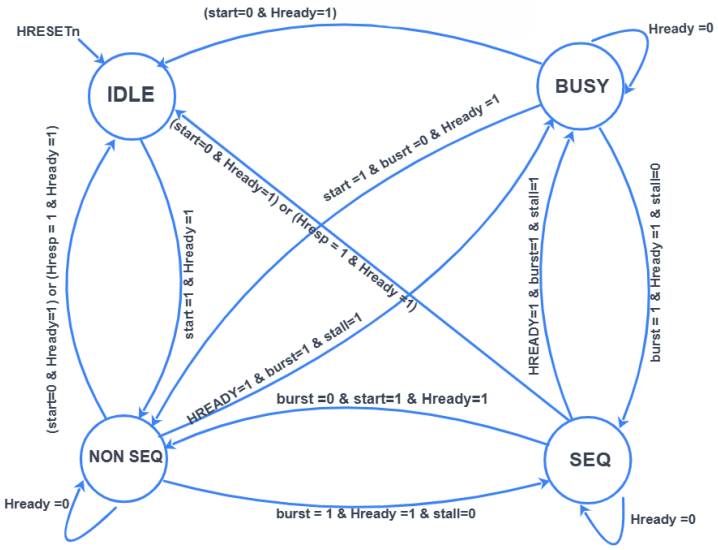
When a slave inserts a number of wait states prior to completing the response, it must drive **HRESP** to OKAY.

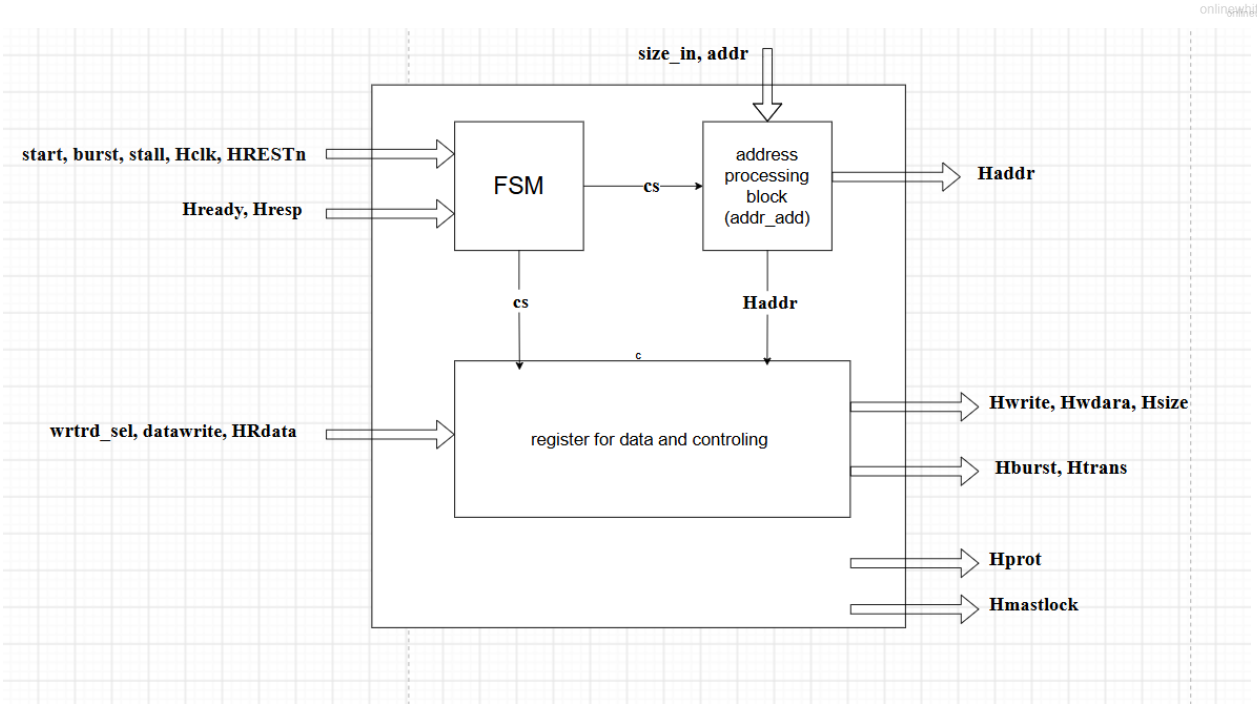
Note

In general, every slave must have a predetermined maximum number of wait states that it inserts before it backs off the bus. This enables you to calculate the latency for accessing the bus.

It is recommended that slaves do not insert more than 16 wait states, to prevent any single access locking the bus for a large number of clock cycles. However, this recommendation is not applicable to some devices, for example, a serial boot ROM. This type of device is usually only accessed during system startup and the impact on system performance is negligible if greater than 16 wait states are used.

Hardware architecture and FSM





Verilog code flow for the master and register bank

The master RTL implements the FSM diagram transitions in the next state logic block by applying case and if conditions. It also implements the output logic and how the outputs will be processed depending on the states and the inputs, and the case statement of the output logic depends on the Htrans that will have its value depending also on the inputs of burst, stall, start, and Hready.

The Hprot and Hmastlock are not supported, so I gave them the default value to get out on the bus.

The addr_add internal wire was added to calculate how much the address will be incremented depending on the size of the data input, so an assign statement was applied to it.

The reg_bank was made to act as a slave to be able to test the transactions of how the Hready, Hresp and HRdata signals will be handled, and the hand shacking between the master and the slave and to be able to test the functionality.

In the RTL of register bank, the Hready and Hresp signals are assigned to values depending on the value of the internal signals in_wait and transfer_starting, from their names they check if the system is in waiting state or not and if not checks if it started transferring or not.

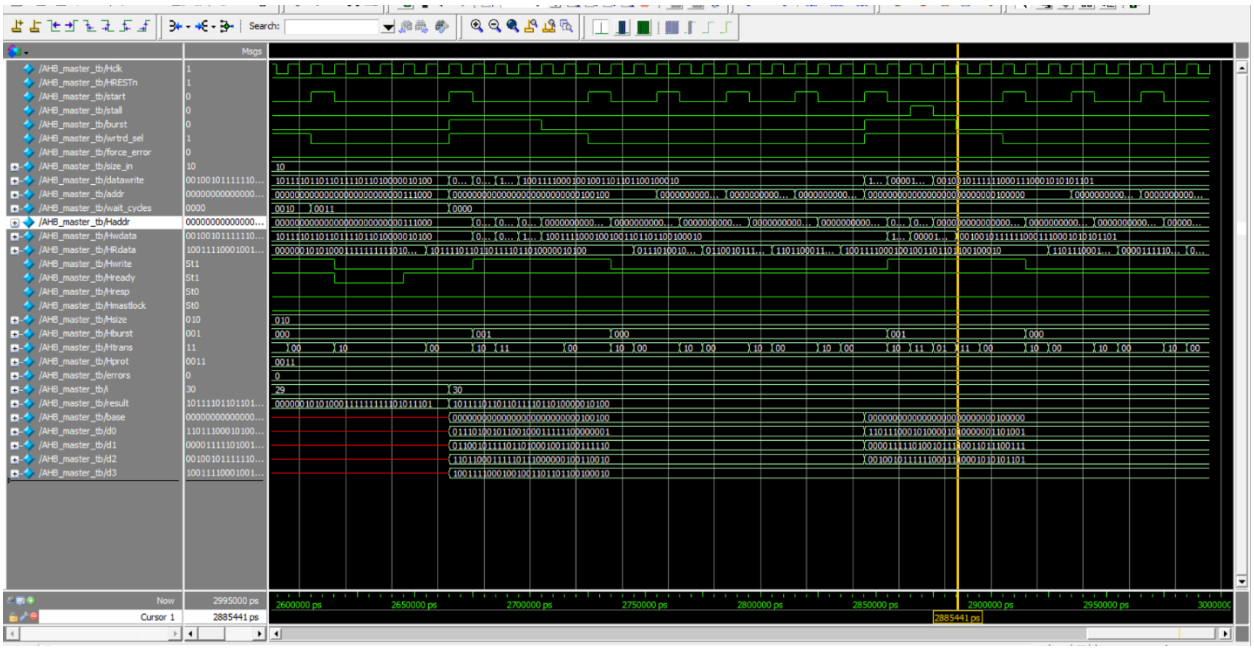
In addition, there are internal signals like count and wait_cycles to count down the required cycles for waiting and when it becomes 0 as this is the last cycle, as illustrated in the standard, the system starts. Also, the resp_errror is checked as if it low so the system is okay and no error, and I force error by the signal force_error to the test cases to check.

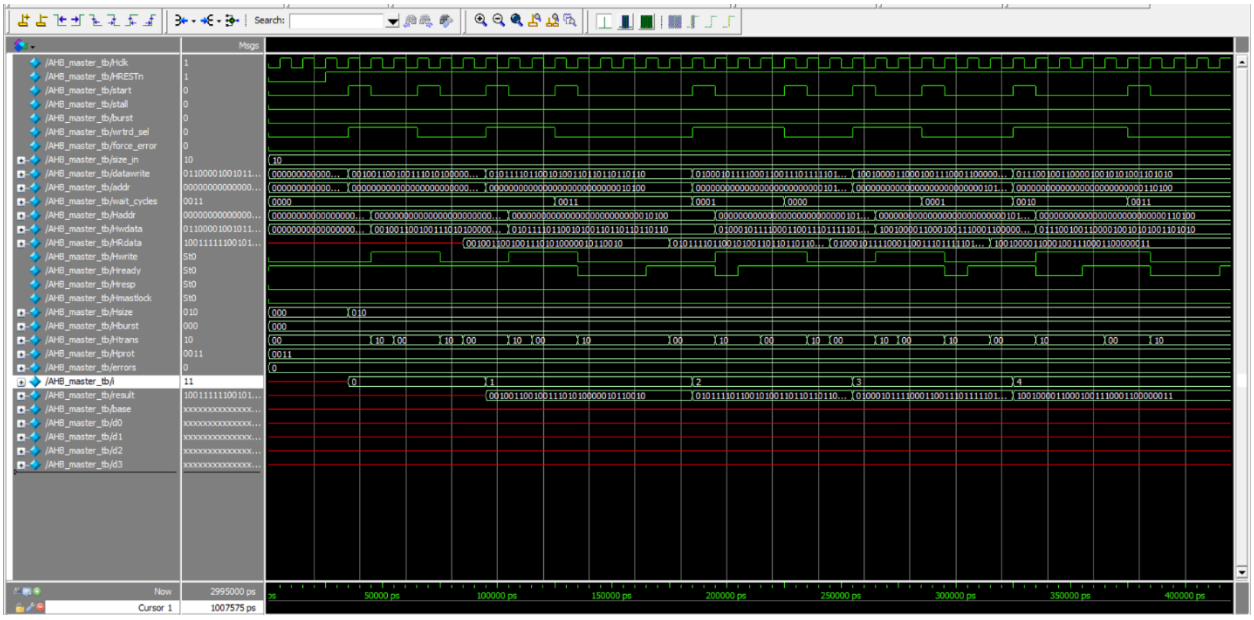
Testbench code flow

The testbench starts with adding some signals like base, d0-d3, errors, results.. to help make a flow of test cases to cover them. Base is a start address randomized number and to save it so when incrementing, the system knows on which address it was, and d values are for saving the written data in many registers, so when we enter datawrtie it takes its data from them. And the wait_cycles is forced and randomized as same as the force_error, so those will deal with the slave logic of hready, hresp, and in_wait to handle the operation needed.

Results:

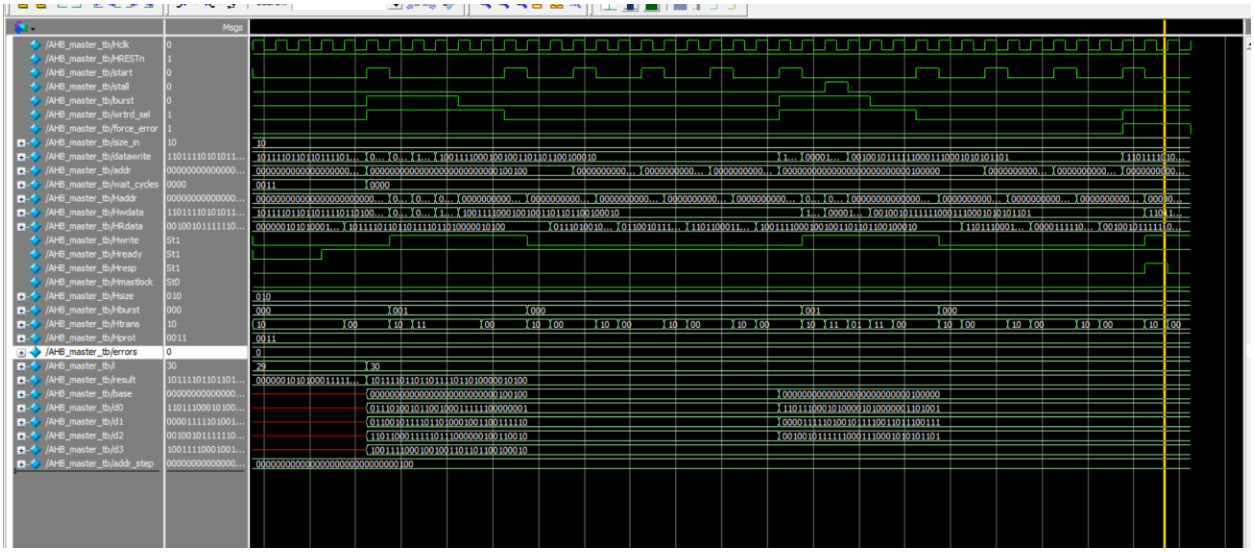
```
sim:/AHB_master_tb/d3
VSIM 3> run -all
# random single write/read
# 4-beat burst write, no stall
# 3-beat burst write with one BUSY beat
# Test complete: 0 mismatches
# ** Note: $stop    : D:/ADI/ADI_AHB/AHB_master_regbank_tb.v(228)
#    Time: 2995 ns  Iteration: 1  Instance: /AHB_master_tb
# Break in Module AHB_master_tb at D:/ADI/ADI_AHB/AHB_master_regbank_tb.v line 228

VSIM 4>
```





The red signals are not indication of error, they are just not activated in those iterations.

The error case:



The Htrans returned to IDLE after Hresp became 1 (error).